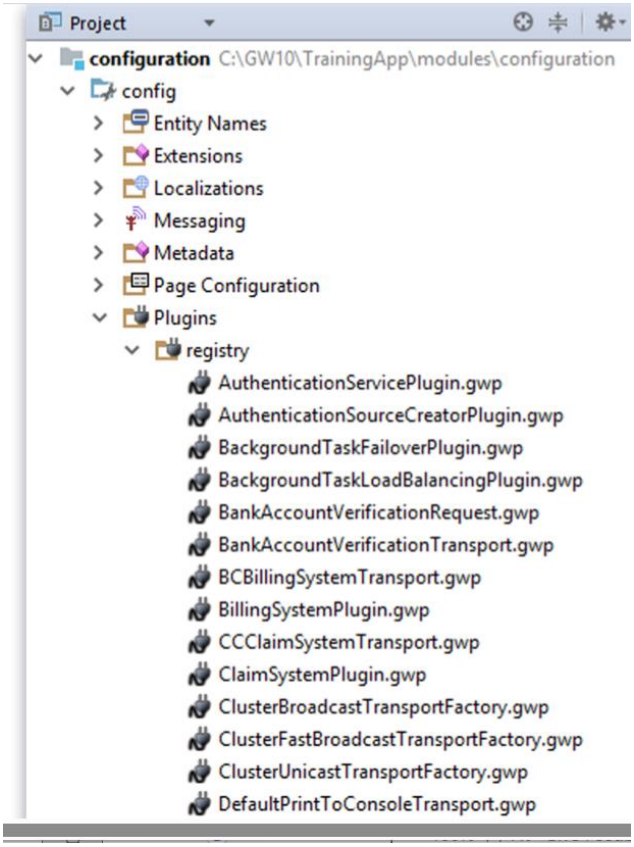
# Plugins

**Introduction to Guidewire plugins**

## Guidewire plugin

- An interface provided by Guidewire
  - Describes methods called by the application
  - May define required application behavior
    - Authentication Plugins
    - Number generators
  - Or may define an integration point
    - Messaging plugins
    - Document Management




- Since you implement the interface(s),
  - You define method behavior
  - Based on the interface contract, you define:
    - Input parameters
    - Return values

Notes

**Guidewire plugin**

In Guidewire, a plugin is a Gosu, OSGi Java, or Java class that contains a set of methods called by internal code to execute application functionality. Guidewire plugins run in the same Java Virtual Machine (JVM) process as the Guidewire application itself.

A plugin might do the required work by itself or it might interact with other external systems. If a plugin must retrieve data from an external system, the integration point must be synchronous. Guidewire strongly recommends that you carefully consider response time, including network response time, as you write your plugin implementations.

In some cases, calls to a plugin block the user from doing further work until the call is complete. End users should not wait for long responses from the application. Developers must ensure that predefined plugins perform quickly and, when possible, synchronously.

# Types of plugins



- Predefined plugins
  - Used to implement customizable behavior for predefined points in the application
- Messaging plugins
  - Used to send messages to external systems and process the resulting replies
- Startable plugins
  - Used to listen for incoming messages from external systems and processes them asynchronously







Notes

**Types of plugins**

Calls to predefined plugins block the user from doing further work. Calls to messaging plugins and startable plugins occur outside of a user context and do not block any user from doing further work.

# Recommended plugin source code options

| Gosu plugin | OSGI plugin | Java plugin |
|---|---|---|
| • Use Guidewire Studio<br>• Native access to application API<br>• Native debugging within Guidewire Studio | • IntelliJ IDEA with OSGi Editor<br>• Access to Guidewire application API<br>• Easily add third party libraries<br>• Easy deployment | • Third party IDE<br>• Needs ALL Guidewire JARs<br>• No inspection support for Guidewire Internal API<br>• Require special deployment |

**Notes**

**Recommended plugin source code options**

**Best practice:**

- Guidewire recommends all new plugins be written in Gosu. The main reasons for this recommendation are: speed of development, code conciseness, and debugging.
- By using Gosu, you can take full advantage of Gosu language features. For example, Gosu blocks, Gosu collection enhancement methods, native access to data model types, intelligent code completion after you type the period (.) character, and type inference. These features help you quickly write concise and easy-to-maintain code.

You can also write your plugin implementations in OSGi or Java.

- An OSGi plugin is a Java plugin that employs OSGi for modularization. Guidewire recommends OSGi for all new Java plugin development. For OSGi plugin development, Guidewire recommends using the IntelliJ IDEA with OSGi Editor. You can learn more about OSGi plugin development in documentation.
- For Java plugin development, there are very specific rules and requirements for class placement in plugins directories. When using a third party IDE for Java plugins, you will need to copy all JAR files from the \java-api\lib\ directory to your Java IDE.

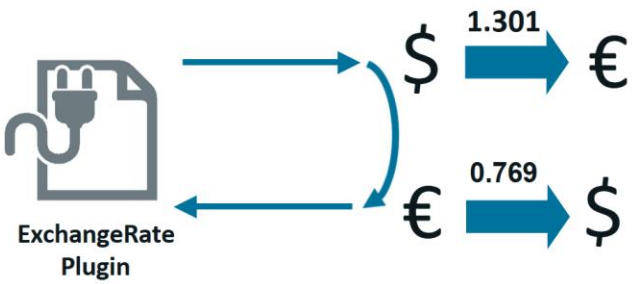
# Use case

**Requirements**

- Create an exchange rate set that includes one exchange rate for every typekey currency

**Design Approach**

1. Determine plugin requirements
2. Write plugin class (implement the interface) in Guidewire Studio
3. Register the plugin by creating or modifying the appropriate plugin registry entry
4. Deploy the changes



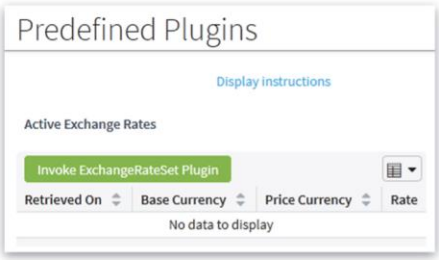

**Use case**

Every predefined plugin meets a specific business need and therefore has unique business-specific issues tied to it.

## Plugin configuration

### Determine Gosu plugin requirements

- What interface(s) must the plugin implement?
  - IExchangeRateSetPlugin
- What methods are included in the interface(s)?
  - createExchangeRateSet
- When are the methods called?
  - By clicking on the **Invoke ExchangeRateSet Plugin** button in the UI
    - Administration → Training: Plugins → Predefined Plugins
- What issues should you consider when implementing the plugin?
  - External system must be online



Notes

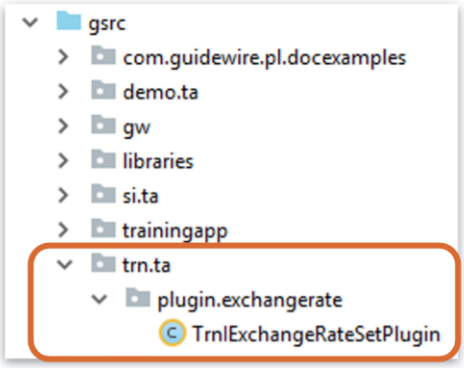
**Determine Gosu plugin requirements**

The Integration Guide for each product describes all the plugins in the plugin overview chapter. In some cases, there are numerous issues around a given plugin, and therefore an entire chapter in the documentation is devoted to it.

In the TrainingApp application, exchange rates are not retrieved automatically. By calling the gw.api.util.CurrencyUtil.invokeMarketExchangeRateSetPlugin method in Gosu code, you can trigger the plugin.

# Plugin class

- Create the package and Gosu class that will implement the plugin
  - Use a package naming convention recommendation
    - `gsrc.<company_code>.<app_code>.plugin.<functional_area>`
  - Use plugin naming convention recommendation
    - `<meaningful_name><interface_implemented>`
    - **TrnlExchangeRateSetPlugin** is a predefined plugin




Notes

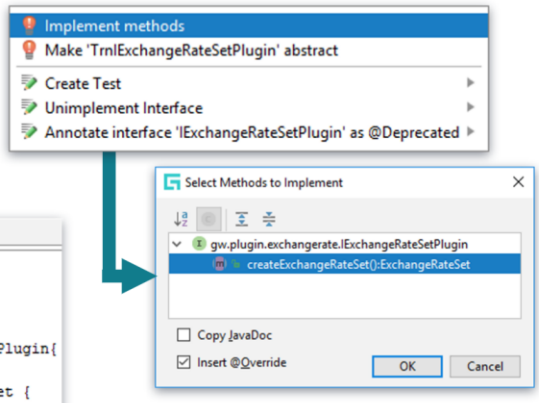
**Plugin class**

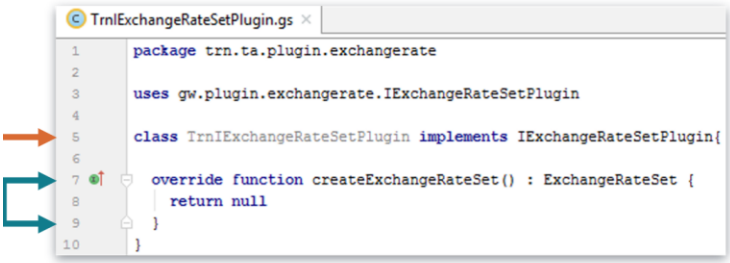
**Best practice:**

- For **predefined plugins**, the recommended naming convention is:
  - <meaningful_name><interface_implemented>
  - Example: SucceedIExchangeRateSetPlugin
- For **messaging plugins**, the recommended naming convention is:
  - <meaningful_name><type_of_message_interface_implemented>
  - Example: BankAccountVerificationTransport
- For **startable plugins**, the recommended naming convention is:
  - <meaningful_name><interface_implemented>
  - Example: InboundJMSIntegrationIStartablePlugin

## Implement required interface(s)

- Implement the interface
  - **ALT+ENTER** to implement methods
  - Every public method from the interface must exist
    - Some methods can be left as stubs that do nothing




**Notes**

**Implement required interface(s)**

**Code example**
**Line 5**
- Implement the exchange rate plugin.

**Lines 7-9**
- Method to implement.

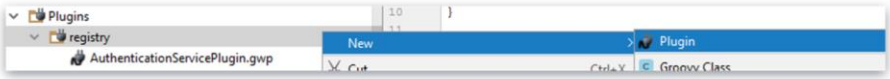
Recall that the first step of plugin implementation is to determine the plugin requirements. This includes the name of the interface to implement, which can be determined from either the Integration Guide or the Java API Reference.

A plugin class can implement one or more interfaces. You must manually type in the **implements** keyword and the name of the appropriate interface. To implement multiple interfaces, list each interface after the implements keyword and separate interfaces with a comma. Click the interface name and press **ALT+ENTER** which adds stubs for all methods that need to be declared in the plugin.

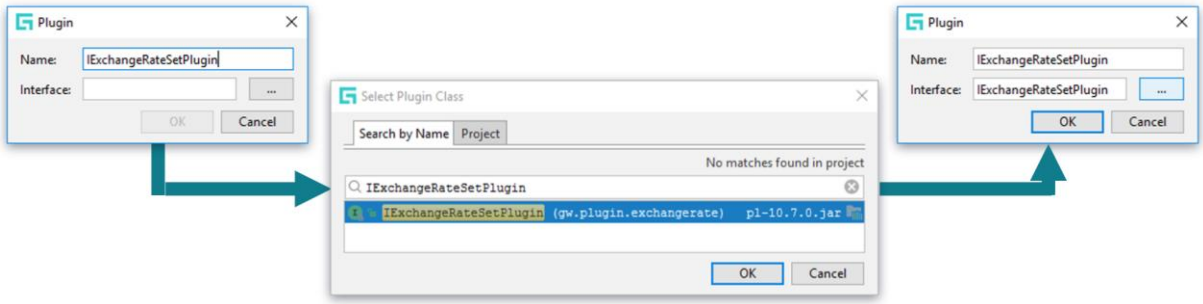
For some plugins, you do not need to override the interface methods. If the method does not return a value, set a property, or call an internal method, you can leave it as it is. For any plugin, you can create and call additional (non-public) methods. For example, an exchange rate plugin might have a private extractRate method which communicates to a SOAP web service to both submit and receive values.

## Create the plugin registry file (if necessary)

- If the plugin definition does not exist, then create a new plugin registry file under `configuration.config.Plugins.registry`




- Specify the name and interface




**Notes**

**Create the plugin registry file (if necessary)**

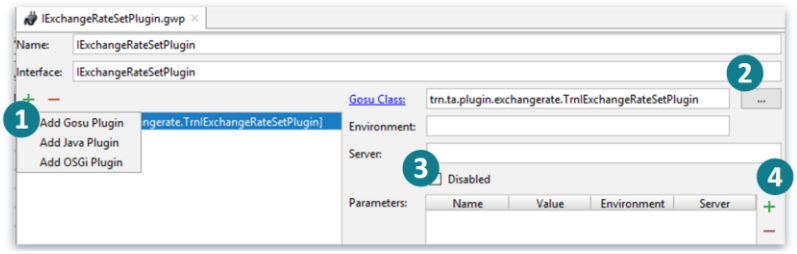
In the Project view, select the registry folder. The path is … config.Plugins.registry.

Each plugin registry is a .gwp file that is an XML document. All plugin files are created in <home_directory>\modules\configuration\config\plugin\registry.

Predefined plugins must be named after the plugin interface. The plugin name and the plugin interface name must be identical for all plugins other than startable or messaging plugins.

# Configure the plugin registry file

- In many cases, the plugin definition already exists in the project
- Plugin editor options
  1. Add plugin type
  2. Specify the plugin class
  3. Disable the plugin
  4. Define plugin parameters

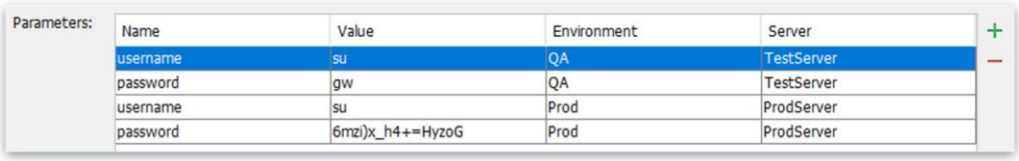


### Configure the plugin registry file

In the slide example, the IExchangeRateSetPlugin is a Gosu plugin. To change the plugin type, first remove the existing plugin reference (click -) and then add a new plugin type (click +) .

For predefined plugins, a given registry file can point to only one class at a time. There can be multiple entries for startable and messaging plugins.

# Plugin parameters




- Specify values at the registry level that can be passed to a plugin class during initialization
  - **Name** and **Value** String fields are required
  - **Environment** and **Server** fields are optional
- Useful for varying plugin behavior without changing the plugin code itself
- Example:
  - Testing environment compared to a production environment
- Plugin class must implement **`InitializablePlugin`** interface and **`set Parameters`** property

### Plugin parameters

Plugin parameters can be useful in the following situations:

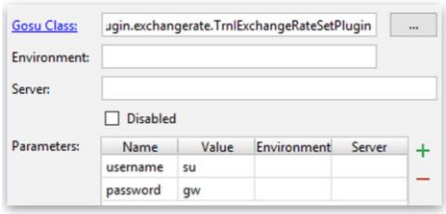
- You want a single plugin implementation to have different behavior over time (such as changing the user name or password needed to access a remote system).
- You want a single plugin implementation to have different behavior in a development environment compared to a production environment.
- You want a single plugin implementation to have different behavior on different servers in a clustered environment.

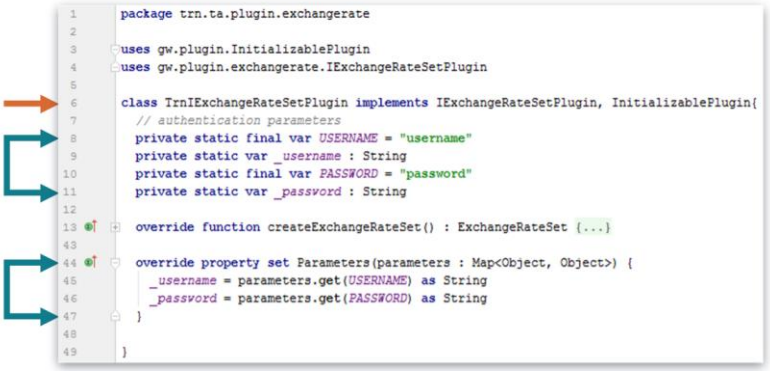
In the Studio Plugins Registry editor, you can add one or more optional parameters to pass to your plugin during initialization. For example, you could use the editor to pass server names, port numbers, timeout values, or other settings to your plugin code. The parameters are pairs of String values, also known as name/value pairs.
You cannot add new or additional configuration parameters to config.xml. If you want to add custom parameters to your configuration of ClaimCenter, consider defining script parameters. You can access the values of script parameters in Gosu code at runtime.

**Global Variables**
*Script parameters* are resources defined in Guidewire Studio™ that you can use as global variables in Gosu code. A script parameter is an application-wide global parameter that has a value that tends to change over time. Script parameters are typically used to set business values that change from year to year. System administrators change the values of script parameters on the Administration tab. Reference Guidewire documentation for more information on how to configure and use script parameters.

# Using plugin parameters

- Add the parameters in the plugin registry
- Reference the parameters in the plugin class



```
1  package trn.ta.plugin.exchangerate
2
3  uses gw.plugin.InitializablePlugin
4  uses gw.plugin.exchangerate.IExchangeRateSetPlugin
5
6  class TrnlExchangeRateSetPlugin implements IExchangeRateSetPlugin, InitializablePlugin{
7    // authentication parameters
8    private static final var USERNAME = "username"
9    private static var _username : String
10   private static final var PASSWORD = "password"
11   private static var _password : String
12
13   override function createExchangeRateSet() : ExchangeRateSet {...}
43
44   override property set Parameters(parameters : Map<Object, Object>) {
45     _username = parameters.get(USERNAME) as String
46     _password = parameters.get(PASSWORD) as String
47   }
48
49 }
```

## Using plugin parameters

**Code example**
**Line 6**
- Implemented the **InitializablePlugin** interface.

**Lines 8-11**
- Add private static variables for each plugin parameter.

**Lines 44-47**
- Implement the **set Parameters** property.
- Call each plugin parameter and set corresponding static variable.

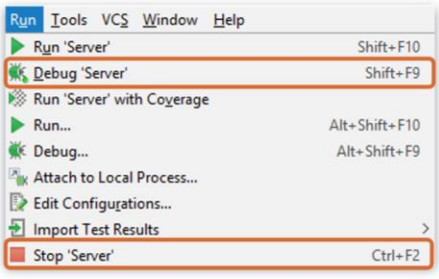
**Best practice:**
- Create private static class variables for each plugin parameter used.
  - **private**: Gosu variables are private by default, but for clarity, the access modifier is added.
  - **static:** Not related to a particular instance. It is shared among all the instances of the class.
  - **final:** Cannot change this value after initialization and this value *must* be initialized.

To make use of plugin parameters, the plugin must implement the InitializablePlugin interface and implement the set Parameters property. The input for this property is a Map object that contains name/value pairs representing the names and values passed in from the plugin registry's Parameters table. The set Parameters property is called only once, before the first use of the plugin.

# Deploy code changes

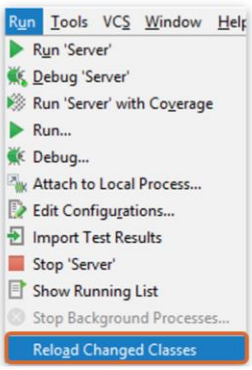
## Restart Server

- New Gosu plugin class
- New or modified plugin registry file




## Reload Changed Classes

- Modified Gosu plugin class



**Identify if each statement is true or false.**

A) Plugins implement methods that are called by internal code.

B) Plugins must be written in Gosu.

C) A plugin class must contain a predetermined set of methods based on the business need it serves.

D) Plugins are used exclusively for external application integration.

A) True

B) False – Guidewire recommends that they be written in Gosu, but they can be written in OSGi or Java.

C) True

D) False – They are also used to execute tasks exclusively within the Guidewire application.

**What is the purpose of a plugin registry entry?**

The plugin registry entry identifies the plugin class to use for a given set of internally called methods and initialization parameters that may be used to affect the plugin's behavior.

**Do you need to restart the server when you:**

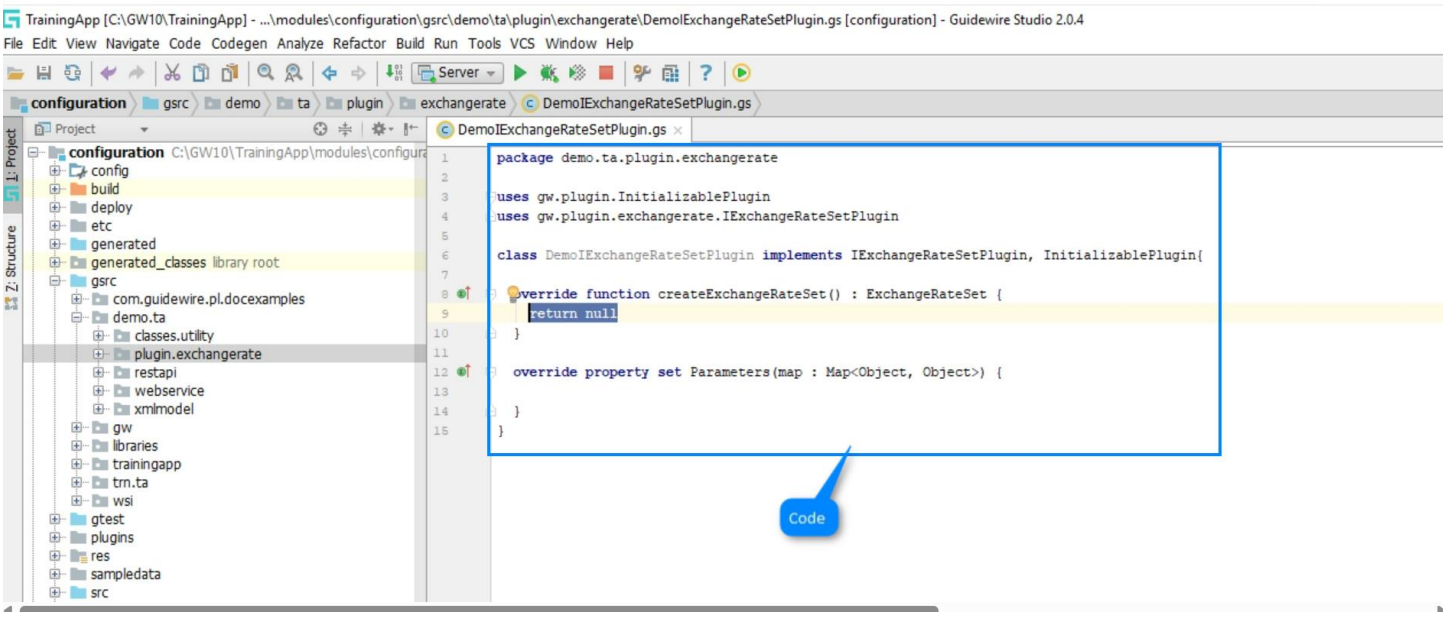
A) Modify code in an existing Gosu plugin?

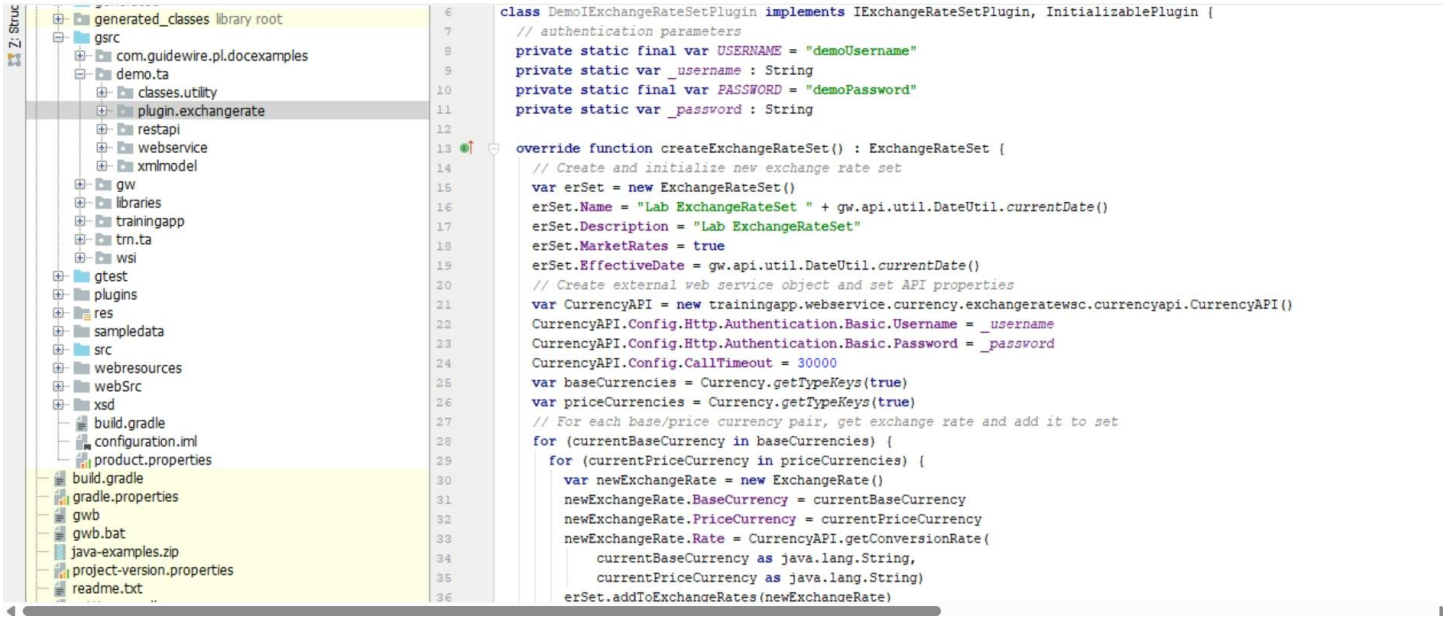
B) Modify a plugin registry entry?

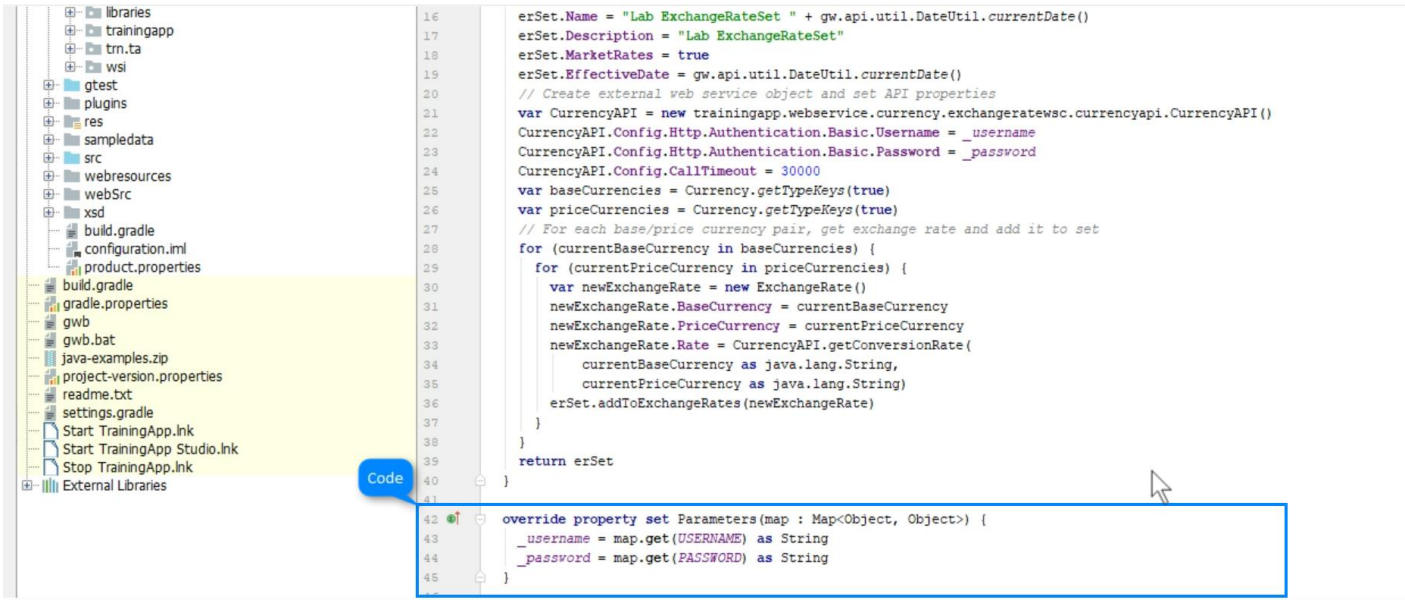
A) No, you do not to restart the server when you make changes to a Gosu plugin class. However, you need to execute Reload Changed Classes.

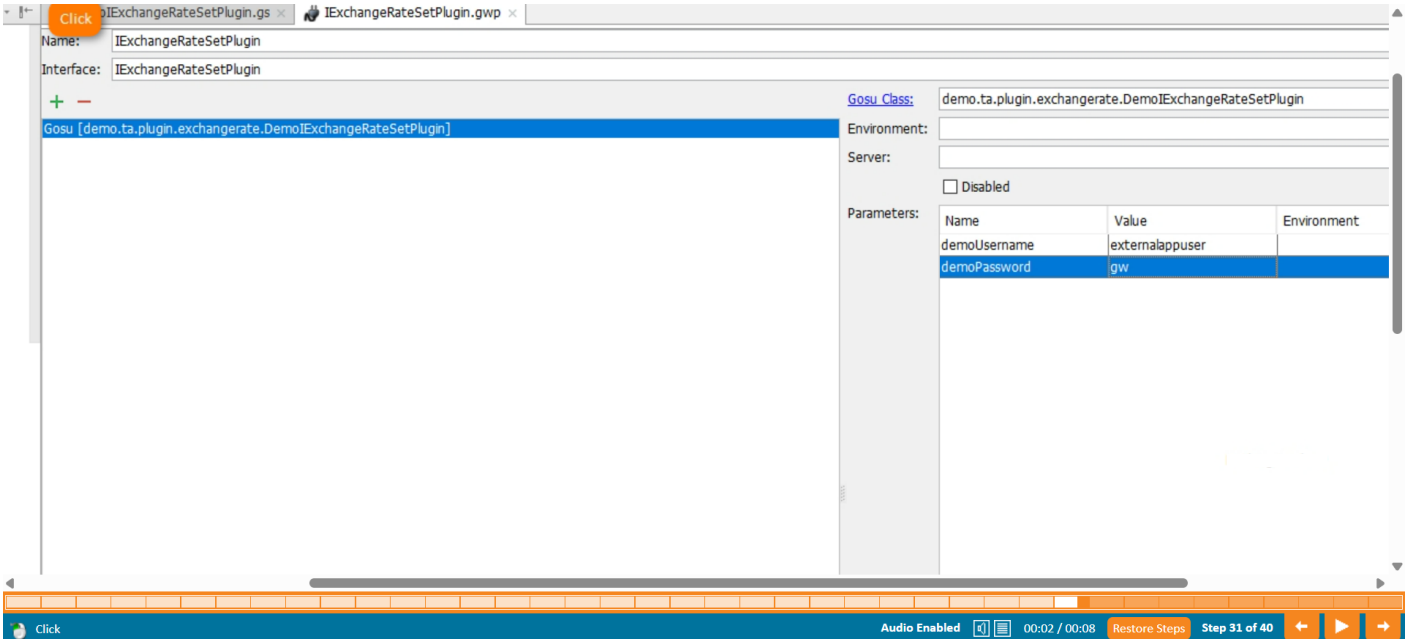
B) Yes, you need to restart the server when modifying a plugin registry entry because plugin registry entries are initialized only during the server application start-up.

Demonstartion


TrainingApp [C:\GW10\TrainingApp] - ...\modules\configuration\gsrc\demo\ta\plugin\exchangerate\DemoIExchangeRateSetPlugin.gs [configuration] - Guidewire Studio 2.0.4
File Edit View Navigate Code Codegen Analyze Refactor Build Run Tools VCS Window Help
Server
configuration gsrc demo ta plugin exchangerate DemoIExchangeRateSetPlugin.gs
Project
DemoIExchangeRateSetPlugin.gs
configuration C:\GW10\TrainingApp\modules\configura
config
build
deploy
etc
generated
generated_classes library root
gsrc
com.guidewire.pl.docexamples
demo.ta
classes.utility
plugin.exchangerate
restapi
webservice
xmlmodel
gw
libraries
trainingapp
trn.ta
wsi
gtest
plugins
res
sampledata
src
1 package demo.ta.plugin.exchangerate
2
3 uses gw.plugin.InitializablePlugin
4 uses gw.plugin.exchangerate.IExchangeRateSetPlugin
5
6 class DemoIExchangeRateSetPlugin implements IExchangeRateSetPlugin, InitializablePlugin{
7
8 override function createExchangeRateSet() : ExchangeRateSet {
9 return null
10 }
11
12 override property set Parameters(map : Map<Object, Object>) {
13
14 }
15 }
Code


generated_classes library root
gsrc
com.guidewire.pl.docexamples
demo.ta
classes.utility
plugin.exchangerate
restapi
webservice
xmlmodel
gw
libraries
trainingapp
trn.ta
wsi
gtest
plugins
res
sampledata
src
webresources
webSrc
xsd
build.gradle
configuration.iml
product.properties
build.gradle
gradle.properties
gwb
gwb.bat
java-examples.zip
project-version.properties
readme.txt
class DemoIExchangeRateSetPlugin implements IExchangeRateSetPlugin, InitializablePlugin {
// authentication parameters
private static final var USERNAME = "demoUsername"
private static var _username : String
private static final var PASSWORD = "demoPassword"
private static var _password : String
override function createExchangeRateSet() : ExchangeRateSet {
// Create and initialize new exchange rate set
var erSet = new ExchangeRateSet()
erSet.Name = "Lab ExchangeRateSet " + gw.api.util.DateUtil.currentDate()
erSet.Description = "Lab ExchangeRateSet"
erSet.MarketRates = true
erSet.EffectiveDate = gw.api.util.DateUtil.currentDate()
// Create external web service object and set API properties
var CurrencyAPI = new trainingapp.webservice.currency.exchangeratewsc.currencyapi.CurrencyAPI()
CurrencyAPI.Config.Http.Authentication.Basic.Username = _username
CurrencyAPI.Config.Http.Authentication.Basic.Password = _password
CurrencyAPI.Config.CallTimeout = 30000
var baseCurrencies = Currency.getTypeKeys(true)
var priceCurrencies = Currency.getTypeKeys(true)
// For each base/price currency pair, get exchange rate and add it to set
for (currentBaseCurrency in baseCurrencies) {
for (currentPriceCurrency in priceCurrencies) {
var newExchangeRate = new ExchangeRate()
newExchangeRate.BaseCurrency = currentBaseCurrency
newExchangeRate.PriceCurrency = currentPriceCurrency
newExchangeRate.Rate = CurrencyAPI.getConversionRate(
currentBaseCurrency as java.lang.String,
currentPriceCurrency as java.lang.String)
erSet.addToExchangeRates(newExchangeRate)


libraries
trainingapp
trn.ta
wsi
gtest
plugins
res
sampledata
src
webresources
webSrc
xsd
build.gradle
configuration.iml
product.properties
build.gradle
gradle.properties
gwb
gwb.bat
java-examples.zip
project-version.properties
readme.txt
settings.gradle
Start TrainingApp.lnk
Start TrainingApp Studio.lnk
Stop TrainingApp.lnk
External Libraries
Code
erSet.Name = "Lab ExchangeRateSet " + gw.api.util.DateUtil.currentDate()
erSet.Description = "Lab ExchangeRateSet"
erSet.MarketRates = true
erSet.EffectiveDate = gw.api.util.DateUtil.currentDate()
// Create external web service object and set API properties
var CurrencyAPI = new trainingapp.webservice.currency.exchangeratewsc.currencyapi.CurrencyAPI()
CurrencyAPI.Config.Http.Authentication.Basic.Username = _username
CurrencyAPI.Config.Http.Authentication.Basic.Password = _password
CurrencyAPI.Config.CallTimeout = 30000
var baseCurrencies = Currency.getTypeKeys(true)
var priceCurrencies = Currency.getTypeKeys(true)
// For each base/price currency pair, get exchange rate and add it to set
for (currentBaseCurrency in baseCurrencies) {
for (currentPriceCurrency in priceCurrencies) {
var newExchangeRate = new ExchangeRate()
newExchangeRate.BaseCurrency = currentBaseCurrency
newExchangeRate.PriceCurrency = currentPriceCurrency
newExchangeRate.Rate = CurrencyAPI.getConversionRate(
currentBaseCurrency as java.lang.String,
currentPriceCurrency as java.lang.String)
erSet.addToExchangeRates(newExchangeRate)
}
}
return erSet
}
override property set Parameters(map : Map<Object, Object>) {
_username = map.get(USERNAME) as String
_password = map.get(PASSWORD) as String
}


Click
IExchangeRateSetPlugin.gwp
Name:
IExchangeRateSetPlugin
Interface:
IExchangeRateSetPlugin
Gosu [demo.ta.plugin.exchangerate.DemoIExchangeRateSetPlugin]
Gosu Class:
demo.ta.plugin.exchangerate.DemoIExchangeRateSetPlugin
Environment:
Server:
Disabled
Parameters:
Name
Value
Environment
demoUsername
externalappuser
demoPassword
gw
Click
Audio Enabled
00:02 / 00:08
Restore Steps
Step 31 of 40